O SATA novo, do zero

Um guia didático: o problema, a régua, o loop, o mapa de ΔC_0, o aliasing, os resultados e o desenho da Nida

Lucas Ayub — verificação do SATA 1D

Setembro 2026

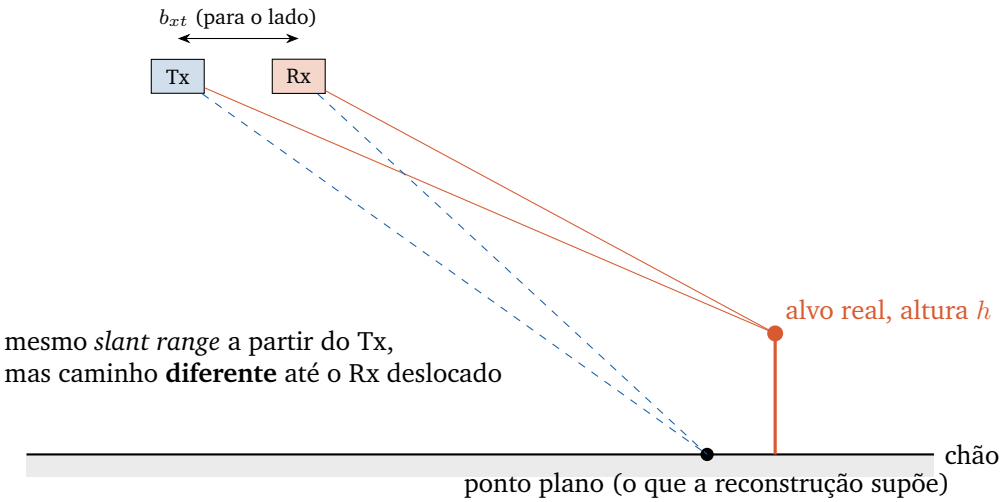
Roteiro

1. **O problema**: por que um alvo com altura estraga a reconstrução multicanal.
2. **O que é o ΔC_0**: o número que mede o estrago, canal por canal.
3. **A ideia do SATA**: olhar o sinal em janelas curtas, onde *frequência = direção = posição*.
4. **A régua** $f_a \leftrightarrow \theta \leftrightarrow x$.
5. **O loop do SATA**, linha por linha do código.
6. **O mapa de ΔC_0**: a única coisa que mudou (antigo $\times$ novo).
7. **Aliasing**, explicado sem medo.
8. **Como ler os gráficos** (cada ponto, cada barra, cada linha).
9. **Resultados**: 1 alvo e 2 alvos.
10. **O desenho da Nida**, comentado.
11. **Limites** e próximos passos. Resumo em um diagrama.

1 O problema: um alvo com altura

O sistema tem **um transmissor (Tx)** e **vários receptores (Rx)**, um atrás do outro ao longo da trajetória (*along-track*, distância b_{at}) e, no nosso caso, também deslocados para o lado (*cross-track*, distância b_{xt}). Cada receptor grava um **canal**. Cada canal sozinho é amostrado devagar ($\mathrm{PRF}_{op} = 500$ Hz); a **reconstrução** junta os 4 canais e monta um sinal como se tivesse sido amostrado 4 vezes mais rápido (2000 Hz).

Para juntar os canais, a reconstrução precisa saber **quanto caminho a mais** a onda percorre até cada receptor. Ela calcula isso supondo que o alvo está **no chão plano** (altura de referência $h_0 = 0$).



O alvo real e o ponto plano estão à **mesma distância** do transmissor (por isso caem na mesma linha de range). Mas, como o receptor está deslocado para o lado, o caminho *alvo* $\rightarrow$ *receptor* é **diferente** para os dois. A reconstrução usa o caminho do ponto plano; o sinal real veio pelo caminho do alvo alto. A diferença é um erro de fase em cada canal.

Ideia-chave

Cada canal carrega um erro de caminho diferente (depende do b_{xt} daquele receptor). Quando a reconstrução junta canais com erros diferentes, ela não consegue cancelar as **ambiguidades** ("fantasmas" do alvo a $\pm 0{,}8$ s). O SATA existe para **remover esse erro de cada canal antes da reconstrução**.

2 O número que mede o estrago: ΔC_0

O pipeline descreve o caminho percorrido (a *range history*) de cada canal por um polinômio. O termo constante, que não depende da frequência, é o C_0. O erro é a diferença entre o C_0 do alvo real e o C_0 que a reconstrução supõe:

$$\boxed{\Delta C_0 \;=\; C_0(\text{alvo real}) \;-\; C_0(\text{referência plana})} \qquad \Longrightarrow \qquad \text{erro de fase} = \frac{2\pi}{\lambda}\,\Delta C_0 .$$

Aproximação útil (para intuição): $\Delta C_0 \approx -\dfrac{b_{xt}\,h}{r\,\tan\theta}$. Cresce com a altura h e com o deslocamento lateral b_{xt} do receptor.

No código (`sar_recon/sata.py`), o valor é calculado com o **mesmo ajuste polinomial** que a reconstrução usa:

```
def residual_C0(cfg, tracks, ptg_real, channel):
    C0_real = GetCoeffNu(ptg_real,      ...channel...)[0]   # alvo verdadeiro
    C0_ref  = GetCoeffNu(cfg.scene.ptg, ...channel...)[0]   # referencia
   plana
    return C0_real - C0_ref                                  # = delta_C0 [m]
```

Exemplo com números

Alvo a $h = 240$ m, $\lambda = 0{,}25$ m, $r_0 = 766$ km. Receptores com $b_{xt} = -150,\ -50,\ +50,\ +150$ m:

canal	0	1	2	3
b_{xt} [m]	-150	-50	$+50$	$+150$
ΔC_0 [cm]	$+12{,}89$	$+4{,}30$	$-4{,}30$	$-12{,}89$
fase $\frac{2\pi}{\lambda}\Delta C_0$	$+371$	$+124$	-124	-371

Poucos centímetros de caminho já viram **centenas de graus** de fase, porque λ é só 25 cm.

3 Por que não basta girar o canal inteiro

Se a cena tivesse **um único alvo**, a correção seria trivial: multiplicar o canal inteiro por $e^{j\frac{2\pi}{\lambda}\Delta C_0}$. Era isso que o mapa *antigo* fazia na prática (o ΔC_0 do alvo repetido na linha toda).

Mas numa cena real há **muitos alvos**, cada um com sua altura, portanto cada um com seu ΔC_0. E o sinal de todos chega **somado** no mesmo canal. Girar o canal inteiro com o valor de um alvo estraga todos os outros.

Analogia

É como uma foto de grupo em que cada pessoa precisa de uma correção de cor diferente. Não dá para aplicar um filtro na foto inteira: é preciso saber **quais pixels são de quem**. O SATA é a ferramenta que descobre "quais pedaços do sinal são de quem" — pela **posição**.

4 A ideia do SATA: frequência = direção = posição

4.1 Por que a frequência diz a posição

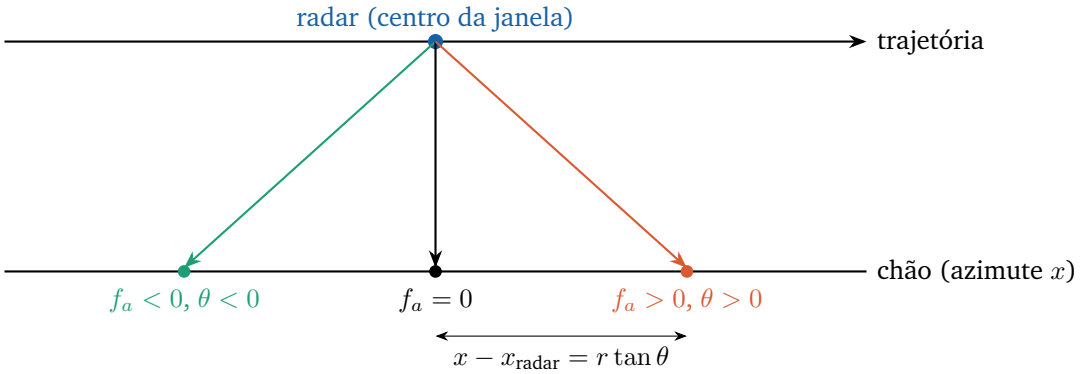
Enquanto o radar passa, a distância até um alvo muda: diminui quando o alvo está à frente, aumenta quando ficou para trás. Isso muda a fase do sinal ao longo do tempo, ou seja, cria uma **frequência** (efeito Doppler):

$$f_a = \frac{2v}{\lambda}\,\sin\theta,$$

onde θ é o ângulo entre "olhar reto para o lado" e "olhar para o alvo".

- alvo **à frente** do radar $\Rightarrow\ \theta > 0\ \Rightarrow\ f_a > 0$;
- alvo **bem ao lado** $\Rightarrow\ \theta = 0\ \Rightarrow\ f_a = 0$;
- alvo **para trás** $\Rightarrow\ \theta < 0\ \Rightarrow\ f_a < 0$.

Se eu pegar um pedaço **curto** do sinal (uma **janela**, ou subabertura) e fizer uma FFT, cada alvo aparece como um **pico** numa frequência. Pela frequência eu sei o ângulo; pelo ângulo e pela posição do radar eu sei **onde no chão** está o alvo.



4.2 A régua $f_a \to \theta \to x$

Para cada bin da FFT (cada frequência f_k), o SATA calcula a célula do chão para onde ele aponta:

$$\theta_k = \arcsin\!\left(\frac{\lambda f_k}{2v}\right), \qquad x_k = \underbrace{\text{centro da janela}}_{\text{onde está o radar}} + \underbrace{\frac{r\tan\theta_k}{v}\,\text{PRF}_{op}}_{\texttt{azpos}_k} = \texttt{posaux}_k.$$

```
betasub = np.arcsin(wl * fsub / (2 * v))          # f -> angulo
azpos   = r * np.tan(betasub) / v * prf           # angulo -> deslocamento [
    amostras]
posaux  = np.round(azpos + center).astype(int)    # + posicao do radar =
    celula no chao
```

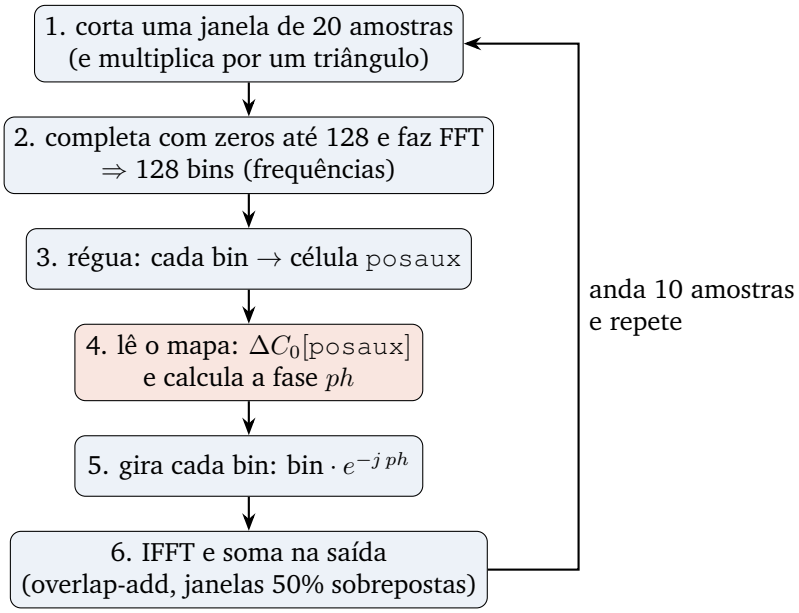
Exemplo com números

$\text{PRF}_{op} = 500$ Hz, $N_{zp} = 128$ bins $\Rightarrow$ um bin a cada $500/128 = 3{,}9$ Hz.

	f_a	θ	x (em torno do radar)
passo entre bins	3,9 Hz	0,002°	3,2 células
alcance total	± 250 Hz	$\pm 0{,}24$	± 203 células

Uma célula $= v/\text{PRF}_{op} \approx 15$ m no chão. Então cada janela "enxerga" o chão de -3 km a $+3$ km em torno do radar.

5 O loop do SATA, passo a passo



O código real (`sar_recon/sata.py`, função `sata_1d`), com os mesmos números:

```
Tsubeff = 20            # janela: 20 amostras  (= sqrt(wl*r/2) no chao, ~309
    m)
Nzp     = 128           # FFT com zero-padding (sata_osf = 4)
hop     = 10            # anda meia janela por vez

for start in range(0, dimx, hop):
    seg  = data[start:start + Tsubeff]                  # passo 1
    buf  = zeros(Nzp); buf[:len(seg)] = seg * win       #   (win = triangulo)
    spec = np.fft.fft(buf)                              # passo 2: 128 bins

    center = start + 0.5 * Tsubeff                      # onde esta o radar
    posaux = np.round(azpos + center).astype(int)       # passo 3: bin ->
    celula

    ph   = -2*np.pi/wl * delta_C0_array[posaux]         # passo 4: LE O MAPA
    spec = spec * np.exp(-1j * ph)                      # passo 5: gira a
    fase

    out[start:start+len(seg)]  += np.fft.ifft(spec)[:len(seg)]   # passo 6
    wsum[start:start+len(seg)] += win[:len(seg)]
out = out / wsum                                        # normaliza a soma
```

Ideia-chave

O kernel **não sabe nada de alvos**. Ele só faz: *bin → célula → lê o valor do mapa naquela célula → gira a fase*. Toda a "inteligência" sobre **quem precisa de qual correção** está no **mapa** `delta_C0_array`. Por isso, a mudança que fizemos foi **só no mapa**; o kernel ficou igual.

Girar a fase não muda o tamanho do bin, só o ângulo:

$$\text{bin} = A\,e^{j\varphi} \quad \longrightarrow \quad A\,e^{j(\varphi - ph)}.$$

Se $ph = 0$ (mapa zero naquela célula), o bin passa intacto.

6 O mapa de ΔC_0: a única coisa que mudou

6.1 Antigo: o valor do alvo na linha inteira

```
# modo "hold" (antigo): um alvo -> o mesmo valor em TODAS as celulas
arr = np.full(naz, dC0_do_alvo)
```

Todo bin de toda janela lê o mesmo valor, **tenha ou não o alvo**. Com um alvo só funciona (bins vazios girados não fazem mal). Com dois ou mais, cada alvo recebe o valor errado em boa parte do tempo.

6.2 Novo: o valor só onde o alvo aparece na STFT

A regra pedida pela Nida: **"o ΔC_0 só deve estar onde o alvo aparece na STFT"**. Como o kernel lê o mapa pela posição, basta colocar o ΔC_0 de cada alvo **exatamente nas células para onde os bins com a energia dele apontam**, e **zero no resto**. Para isso o código precisa prever três coisas:

ingrediente	pergunta que responde	valor aqui
(a) posição x_{alvo}	em que célula o alvo está, neste canal?	1547 (canal 3)
(b) largura hw	quantas células o pico dele ocupa?	± 82 células
(c) aliasing X, M	onde mais ele aparece "disfarçado"?	± 405 células, 1 de cada lado

O mapa vira um conjunto de **barras**: ΔC_0 em $[x_{\text{alvo}}+mX-hw,\ x_{\text{alvo}}+mX+hw]$ para $m = -1, 0, +1$; zero fora.

```
def build_delta_C0_array(cfg, tracks, channel, mode="footprint", ...):
    for (dx, dy, dh) in cfg.scene.extra_offsets:           # cada alvo
        valor = residual_C0(cfg, tracks, alvo, channel)     # seu delta_C0
        pix   = az_pixel_of_scatterer(cfg, dx, channel)     # (a) posicao
        by_pixel[pix].append(valor)
    hw   = sata_footprint_halfwidth(cfg)                    # (b) largura
    X, M = sata_alias_images(cfg)                           # (c) aliasing
    return footprint_map(by_pixel, naz, hw, X, M)           # barras; resto =
     0
```

(a) Posição do alvo em cada canal

O eixo de tempo é centrado (radar em $x = 0$ na amostra $N_{a,ch}/2$) e cada amostra do canal anda $v/\text{PRF}_{op} \approx 15{,}4$ m. O receptor k fica $b_{at,k}$ atrás do transmissor, então o "centro de fase" do par Tx–Rx fica $b_{at,k}/2$ atrás:

$$x_{\text{alvo}} = \frac{N_{a,ch}}{2} + \frac{x_0 + dx + b_{at,k}/2}{v/\text{PRF}_{op}}.$$

```
def az_pixel_of_scatterer(cfg, dx, channel):
    ds    = cfg.system.vs / cfg.PRF_op
    x_abs = cfg.scene.x0 + dx + 0.5 * cfg.array.bat[channel]
    return int(round(cfg.Na_ch / 2 + x_abs / ds))
```

(A versão antiga ignorava x_0 e $b_{at}/2$: até $\sim$10 células de erro no canal 3.)

(b) Largura: quanto o pico ocupa

A janela é curta (20 amostras, com um triângulo). Por isso um alvo não vira um "risquinho" na FFT, mas um **morrinho**: o espectro de um triângulo (um sinc^2). Ele tem um lóbulo principal e lóbulos laterais:

	principal	1º lateral	2º lateral	3º lateral
nível	0 dB	$-26{,}5$ dB	$-35{,}7$ dB	$-41{,}6$ dB
termina em (nulo)	50 Hz	100 Hz	150 Hz	200 Hz

A barra cobre até o **2º nulo** ($4\,\mathrm{PRF}_{op}/T_{sub,eff} = 100$ Hz): lóbulo principal + primeiros laterais, ou seja, toda a energia acima de ~ -30 dB. Passando 100 Hz pela régua: $hw \approx 82$ células.

```
def sata_footprint_halfwidth(cfg):
    Tsubeff = round(sqrt(wl*r/2) * prf / v * 0.5) * 2     # 20 (igual ao
   sata_1d)
    f_edge  = 4.0 * prf / Tsubeff                          # 2o nulo = 100 Hz
    return ceil(r * tan(arcsin(wl*f_edge/(2*v))) / v * prf)    # -> 82 celulas
```

(c) Aliasing: a próxima seção inteira

7 Aliasing, sem medo

Analogia: a roda da carroça no filme

Num filme a 24 quadros por segundo, a roda de uma carroça rápida parece girar **devagar** ou até **para trás**. A câmera amostra devagar demais para a velocidade real, e o movimento aparece "dobrado" para uma velocidade falsa. Com frequências é igual: se a frequência real passa de metade da taxa de amostragem, ela aparece como outra frequência.

Cada canal é amostrado a $\mathrm{PRF}_{op} = 500$ Hz, então só consegue representar frequências entre -250 e $+250$ Hz. Mas, ao longo da abertura, a Doppler do alvo vai de -640 a $+640$ Hz (banda da antena $B = 4v\sin(\theta_{tx}/2)/\lambda = 1281$ Hz). Quando a Doppler verdadeira passa de 250 Hz, ela aparece **dobrada**:

$$f_{\text{aparente}} = f_{\text{verdadeira}} - m \cdot \mathrm{PRF}_{op}.$$

O alvo **aparece de verdade na STFT**, só que com a frequência errada. A régua então manda esse pico para uma **célula errada**:

$$x_{\text{aparente}} \approx x_{\text{alvo}} - m\,X, \qquad X = \frac{r\tan\!\left(\arcsin\frac{\lambda\,\mathrm{PRF}_{op}}{2v}\right)}{v}\,\mathrm{PRF}_{op} \approx 405 \text{ células.}$$

Como X não depende da janela, o alvo aliased sempre cai no mesmo lugar: $x_{\text{alvo}} \pm 405$. Número de dobras: $M = \lceil (640 - 250)/500 \rceil = 1$ de cada lado.

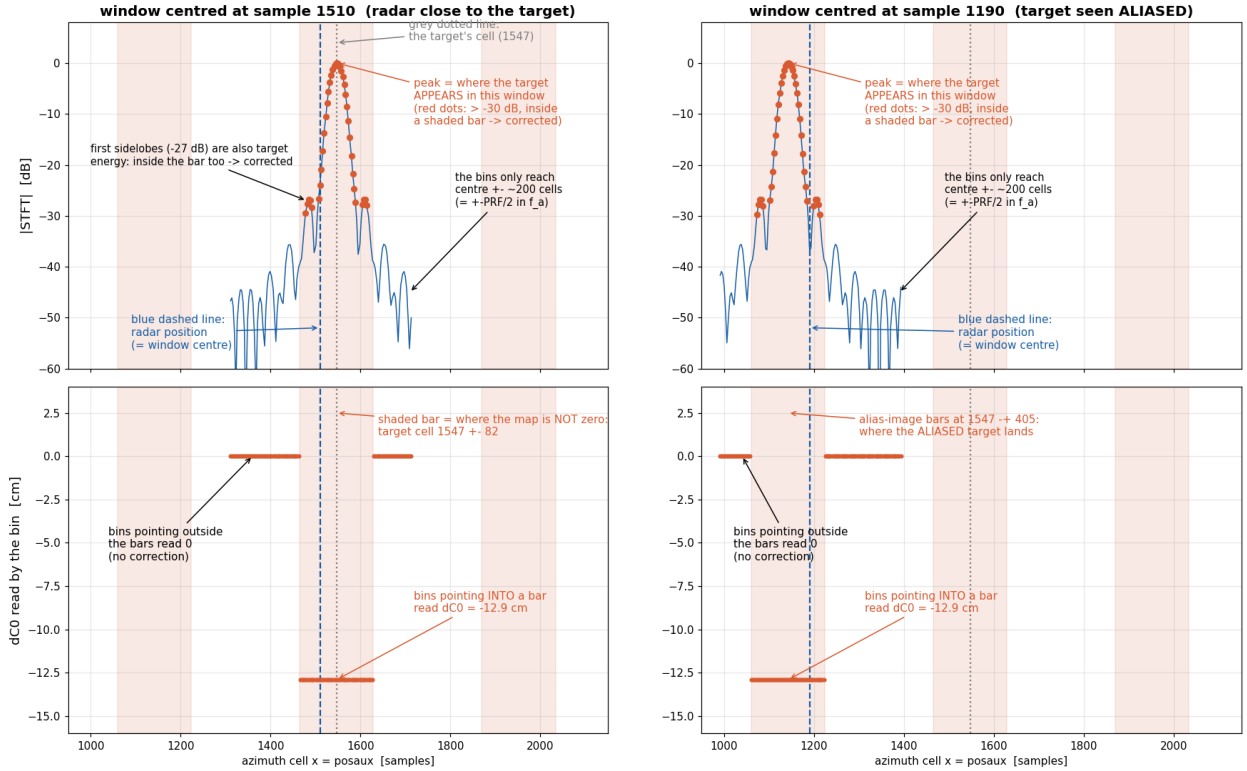
Exemplo com números: janela com centro em 1190

Radar em 1190, alvo em 1547 (357 células à frente). A Doppler verdadeira seria $\approx +441$ Hz, que passa de 250. Ela aparece como $441 - 500 = -59$ Hz. A régua leva -59 Hz para $1190 - 48 = \mathbf{1142} = 1547 - 405$. Por isso existe uma barra em 1142: para que essa energia do alvo, que apareceu "no endereço errado", também receba o ΔC_0 dele.

```
def sata_alias_images(cfg):
    X = r * tan(arcsin(wl*prf/(2*v))) / v * prf        # 405 celulas por "
   dobra"
    B = 4 * v * sin(theta_tx/2) / wl                   # banda Doppler: 1281
   Hz
    M = max(0, ceil((B/2 - prf/2) / prf))              # dobras de cada lado:
     1
    return X, M
```

8 Como ler os gráficos

Cada página dos PDFs de teste é **uma janela** do loop. As três colunas mostram **os mesmos 128 bins** com três "nomes" no eixo horizontal: θ, f_a e x. A coluna da direita (x = `posaux`) é a mais útil. A figura abaixo mostra duas janelas do teste de 1 alvo, anotadas.



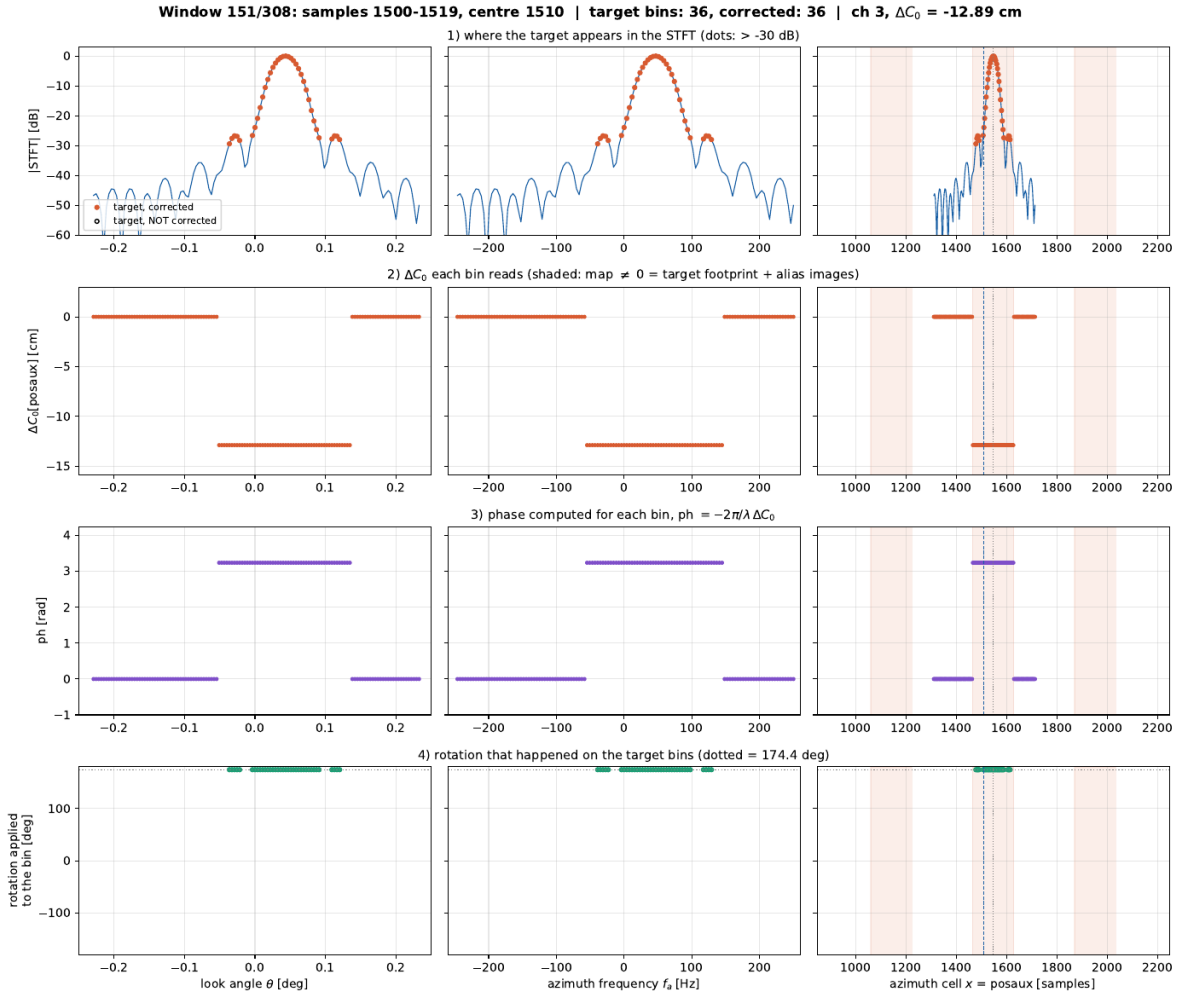
elemento	significado
linha azul (em cima)	a STFT da janela: 128 bins, cada um desenhado na célula para onde aponta
tracejado azul vertical	posição do radar = centro da janela. Os bins só alcançam centro $\pm$ 203 células
pontilhado cinza/colorido	a célula verdadeira do(s) alvo(s)
barras sombreadas	onde o mapa **não é zero**: alvo ± 82 e suas imagens de alias a ± 405. Bin que aponta para dentro recebe ΔC_0; fora, recebe 0
pontos cheios	bins onde o alvo **aparece** (acima de -30 dB) **e** que recebem o ΔC_0 dele: acerto
círculos vazios	bins onde o alvo aparece mas **não** recebem o ΔC_0 dele: falha
pontos de baixo	o valor que cada bin lê do mapa ("degraus" em 0 e em ΔC_0)
rotação medida	$\arg$(bin depois) $-$ $\arg$(bin antes): quanto a fase do bin foi girada de fato

Atenção: o -30 dB

O limiar de -30 dB é **só para desenhar e contar** os pontos (conferir a previsão). O SATA **não usa limiar nenhum**: ele usa as barras, que vêm da geometria (posição, largura, alias). O -30 dB foi escolhido porque separa o lóbulo principal + 1º lateral ($-26{,}5$ dB) do resto, que é a mesma região que a barra cobre.

9 Resultado: 1 alvo

Cena: um alvo a 240 m na célula 1547, canal 3 ($\Delta C_0 = -12{,}89$ cm, fase ideal $174{,}4$ módulo 360°). Iluminado entre as amostras 1038 e 2056. Tudo pelo pipeline (`make_topo_config`, `getRawData1D`, `build_delta_C0_array`, `sata_1d`).

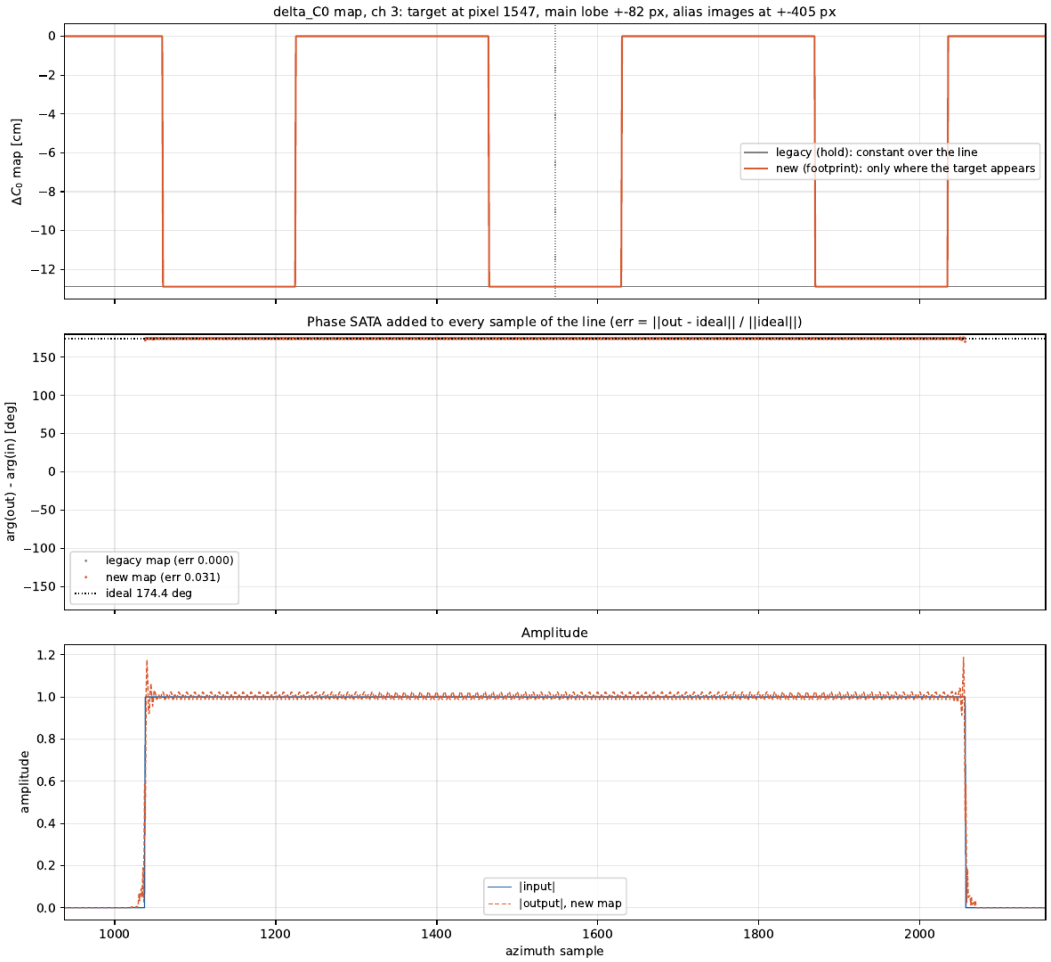


Janela 151 (radar em 1510, perto do alvo). Linha 1: o pico cai na barra central, todos os pontos cheios. Linha 2: os bins que apontam para a barra leem $-12{,}89$ cm, os outros 0. Linha 3: a fase aplicada. Linha 4: a rotação medida nos bins do alvo é exatamente $174{,}4$.

O que acontece ao passar as páginas:

1. **radar bem antes do alvo**: o alvo aparece *aliased*, pico na barra da esquerda ($\sim$1142) — corrigido;
2. **transição**: a Doppler cruza ± 250 Hz; o pico aparece partido nas duas pontas do eixo f_a, cada metade numa barra;

3. **radar perto do alvo**: pico direto na barra central — corrigido;
4. **radar depois do alvo**: aliased para o outro lado, barra da direita (∼1952) — corrigido.

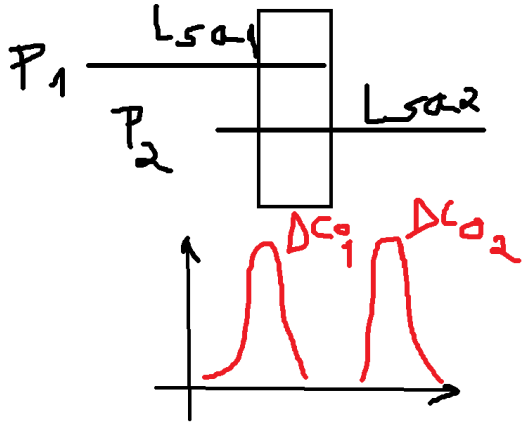


Última página: em cima, o mapa antigo (constante) e o novo (três barras). No meio, a fase que o SATA adicionou a cada amostra do sinal: $\approx 174{,}4$ ao longo de toda a abertura.

bins onde o alvo aparece (todas as janelas)	4048
desses, com o ΔC_0 dele	**96,2%**
erro da linha final vs. correção ideal	**3,1%**

Os 3,1% vêm das janelas nas **pontas da abertura** (pegam só parte do sinal) e de alguns bins na **transição do alias**. O mapa antigo dá 0% aqui só porque não há mais nada na cena.

10 O desenho da Nida



O que o desenho diz:

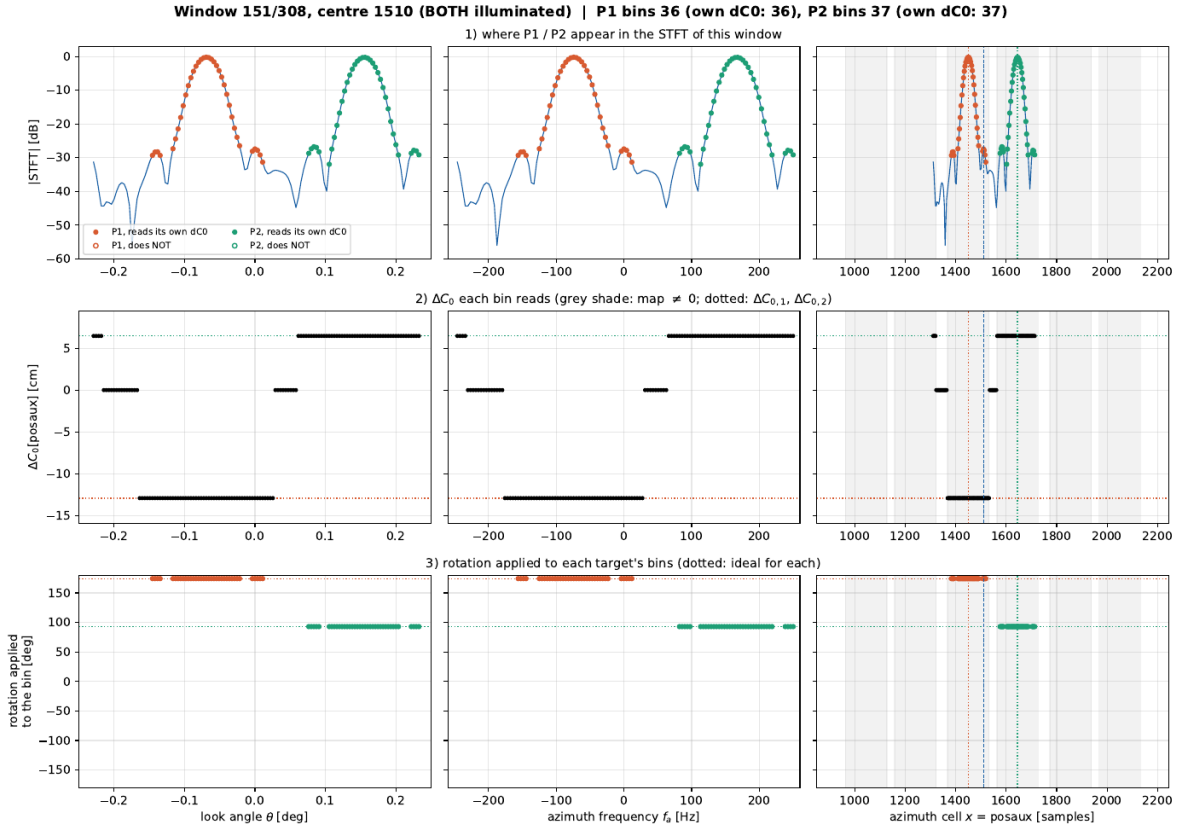
- P_1 e P_2 são dois alvos; as linhas L_{sa1} e L_{sa2} são as suas **aberturas sintéticas**: o trecho da passagem do radar em que cada um está iluminado.
- O **retângulo** é uma janela do SATA num momento em que **os dois** estão iluminados (a interseção).
- Nessa janela, cada alvo é visto num **ângulo diferente** (o radar está numa posição diferente em relação a cada um), então a STFT mostra **dois picos** em f_a diferentes (as curvas vermelhas).
- Pela régua $f_a \leftrightarrow \theta \leftrightarrow x$, cada pico cai **na posição do seu alvo**. Então o mapa deve ter $\Delta C_{0,1}$ na região de P_1 e $\Delta C_{0,2}$ na de P_2, e cada pico é corrigido com o seu valor **na mesma janela**.

Ideia-chave: o desenho é exatamente o mapa novo

O modo `footprint` faz isso: uma barra por alvo, centrada na posição dele, com o seu ΔC_0, e zero no resto. O kernel continua igual; ele só lê o mapa na célula de cada bin.

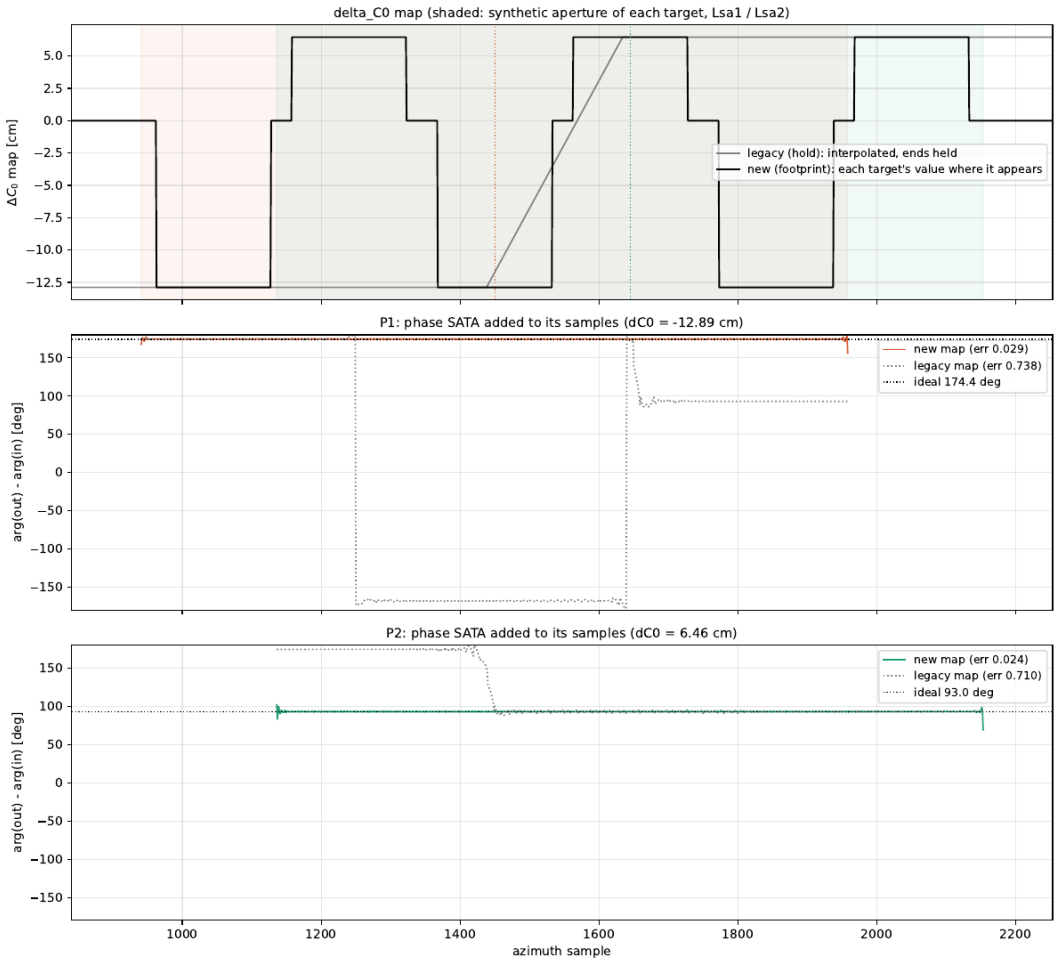
O teste do desenho

P_1: 240 m, célula 1450, $\Delta C_{0,1} = -12{,}89$ cm. P_2: -120 m, célula 1645, $\Delta C_{0,2} = +6{,}46$ cm. Separados por 3 km. Aberturas: P_1 de 941 a 1958, P_2 de 1136 a 2153; **interseção de 1136 a 1958** (o retângulo).



Janela 151 (radar em 1510), dentro da interseção. Linha 1: **dois picos**, P_1 (laranja) em $f_a \approx -70$ Hz (está 60 células atrás do radar) e P_2 (verde) em $f_a \approx +170$ Hz (135 células à frente). No painel x, cada pico cai sobre a linha pontilhada do seu alvo, e todos os pontos estão cheios. Linha 2: três degraus, $-12{,}89$ / 0 / $+6{,}46$. Linha 3: bins de P_1 girados $174{,}4$, bins de P_2 girados $93{,}0$ — cada um com a sua rotação ideal.

	P_1	P_2
bins onde aparece	3964	3999
leem o próprio ΔC_0	**96,8%**	**97,2%**
erro vs. ideal, mapa **novo**	**2,9%**	**2,4%**
erro vs. ideal, mapa **antigo**	73,8%	71,0%



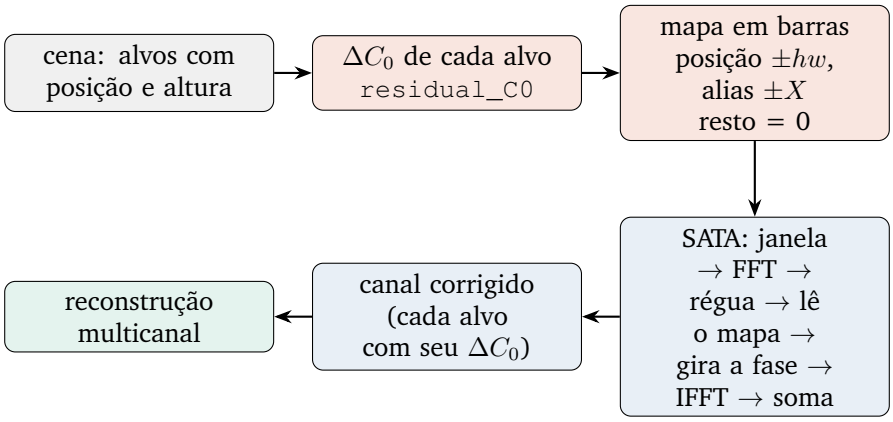
Última página: em cima, o mapa novo (6 barras: cada alvo e suas duas imagens de alias) e o antigo (uma rampa entre os dois valores, mantida nas pontas). Embaixo, a fase adicionada a cada alvo: com o mapa novo, P_1 fica em $174{,}4$ e P_2 em $93{,}0$ o tempo todo; com o antigo, P_1 passa a receber a correção de P_2 no meio da abertura.

Quando o desenho deixa de funcionar

O desenho supõe que os **dois picos estão separados** na STFT. Isso depende da distância entre os alvos em azimute:

distância	o que acontece
$>$ 2,5 km	barras separadas: cada alvo se comporta como alvo único (o teste acima)
$\sim$0,6 a 2,5 km	picos separados, mas os lóbulos laterais de um invadem a barra do outro: pequeno erro
$< \delta x \approx$ 0,3–0,6 km	**os picos se fundem**: os dois alvos ocupam os mesmos bins, e um bin só aceita uma fase. É o limite de resolução do SATA 1D. O código atual dá o maior $\lvert\Delta C_0\rvert$; o IDL original daria, na prática, a média da topografia nessa faixa
$\approx$ 6,2 km ($\pm$1,3)	**colisão de alias**: a imagem aliased de um cai sobre a posição do outro, e o mapa só tem um valor ali. Só as sub-bandas separam os dois

11 Resumo em um diagrama



Ideia-chave: em três frases

(1) Numa janela curta, a frequência de um bin diz **para onde no chão** ele aponta. (2) O mapa diz **qual correção** cada lugar do chão precisa: o ΔC_0 do alvo onde o alvo aparece (direto ou aliased) e zero no resto. (3) O SATA gira cada bin pela correção do lugar para onde ele aponta; assim, cada alvo recebe o seu ΔC_0, mesmo com vários alvos no mesmo canal.

12 Limites e próximos passos

- **Resolução**: variações de altura menores que $\delta x = \sqrt{\lambda r/2} \approx 309$ m não são acomodadas (o próprio kernel imprime isso).
- **Colisão de alias**: com muitos alvos, imagens aliased podem cair sobre outros alvos; a solução é o SATA por **sub-banda**, onde a frequência não é ambígua.
- **Critério dentro da resolução**: hoje é "vence o maior $|\Delta C_0|$"; alternativas são a média ponderada pela energia ou um perfil de terreno (como o DEM do IDL).
- **Próximos testes**: dois alvos dentro de δx e em colisão de alias (basta mudar `TARGETS` no script); depois, medir as ambiguidades após a reconstrução.

Onde está cada coisa

arquivo	conteúdo
`sar_recon/sata.py`	`sata_1d` (kernel, inalterado); `build_delta_C0_array` (novo modo `footprint`, padrão; `mode="hold"` = antigo); `az_pixel_of_scatterer`, `sata_footprint_halfwidth`, `sata_alias_images`, `footprint_map`
`sar_recon/subband_recon.py`	mesma regra no mapa por sub-banda
`plot_sata_correction_windows.py`	(pasta `sata1d_verification`) teste de 1 alvo, uma página por janela
`plot_sata_two_targets_windows.py`	teste de 2 alvos (o desenho)

Glossário rápido

canal	o sinal gravado por um receptor.
PRF_{op}	taxa de amostragem de cada canal (500 Hz).
janela / subabertura	pedaço curto do sinal (20 amostras) onde o SATA faz a FFT.
bin	uma das 128 frequências da FFT da janela.
`posaux`	a célula do chão para onde um bin aponta.
ΔC_0	caminho extra (em metros) que um alvo com altura tem num canal, em relação ao que a reconstrução supõe.
mapa `delta_C0_array`	a tabela "célula $\rightarrow$ correção" que o SATA consulta.
aliasing	frequência alta demais para a amostragem, que aparece "dobrada" como outra frequência.
δx	resolução do SATA no chão (≈ 309 m).